Technical Report

Bio-Sync HPC: A Cross-Layer Self-Optimising Federated Learning Infrastructure for Biomedical IoT

Submitted by: [**Sanchit Kumar Dogra**]

Roll No.: **IMT2022035**

Date: **March 29, 2026**

Project Summary in Plain Terms

This project builds and deploys a production-grade, event-driven infrastructure for privacy-preserving machine learning on biomedical signals. Two simulated hospitals stream high-frequency EEG data through a Federated Learning pipeline deployed on Microsoft Azure Kubernetes Service. The system intelligently prunes redundant data at the edge using frequency-domain analysis, trains a neural network without ever sharing raw patient data, scales compute resources automatically when seizure events are detected, and maintains a live digital mirror of patient state in Azure Digital Twins — all while providing formal differential privacy guarantees compliant with HIPAA and GDPR.

Contents

1 Problem Statement and Motivation **3**

2 System Architecture **3**
2.1 Overview . . . 3
2.2 Component Inventory . . . 4
2.3 Data Flow . . . 4

3 CNE Upgrade: Bidirectional gRPC Streaming **5**
3.1 Problem with Unary RPC . . . 5
3.2 Bidirectional Streaming Solution . . . 5
3.3 ADT Debounce . . . 6

4 ECE Upgrade: FFT-Based Spectral Edge Filter **6**
4.1 Limitation of Amplitude Thresholding . . . 6
4.2 Frequency Band Physiology . . . 6
4.3 FFT Filter Implementation . . . 7
4.4 Bandwidth Reduction . . . 8

5 HPC Upgrade: KEDA Event-Driven Autoscaling **8**
5.1 Scaling Architecture . . . 8
5.2 Multi-Tenant Network Isolation . . . 8

6 MLOps Upgrade: Real Federated Learning **9**
6.1 From Simulation to Real Backpropagation . . . 9
6.2 Weighted Federated Averaging . . . 9
6.3 Convergence Evidence . . . 10

7 Security Upgrade: Formal Differential Privacy **10**
7.1 Limitations of Naive Laplacian Noise . . . 10
7.2 Opacus Formal DP Mechanism . . . 11
7.3 Privacy Audit Trail . . . 11

8 Infrastructure Design Decisions **12**
8.1 AKS over Minikube . . . 12
8.2 Static Azure File PVC . . . 12
8.3 MLflow Version Pinning . . . 12
8.4 JSON Patch Operation . . . 12

9 Measured Results **13**

10 Dashboard and Observability **13**
10.1 Architecture . . . 13
10.2 Five Live Panels . . . 14

11 Known Limitations and Honest Assessment **14**
11.1 What This Project Is . . . 14
11.2 Model Performance Limitation . . . 14
11.3 What Can Be Honestly Claimed . . . 15

12 Future Work **15**

13 Conclusions **16**

1. Problem Statement and Motivation

High-frequency biomedical signals such as electroencephalography (EEG) present a fundamental tension between clinical utility and practical deployability. A standard clinical EEG device samples at 256 Hz, generating approximately 36 MB of data per hour per patient. Streaming this continuously to a cloud inference server is impractical for three independent reasons.

First, **power consumption**: wearable edge devices have limited battery capacity, and continuous radio transmission is the dominant power draw. Second, **network cost**: cloud egress pricing at scale makes continuous streaming economically unviable. Third, **regulatory compliance**: raw EEG data constitutes protected health information under HIPAA and sensitive personal data under GDPR, and transmitting it to a centralised server creates a data breach surface that is difficult to defend.

Standard Federated Learning (FL) frameworks address the third concern but not the first two: they still require continuous edge-to-cloud communication, simply of model gradients rather than raw data. Furthermore, existing FL systems operate on fixed schedules, which is inappropriate for biomedical signals where clinically important events (seizures, arrhythmias) are rare and bursty rather than continuous.

Bio-Sync HPC addresses all three problems through a cross-layer design that integrates signal processing at the edge, event-driven compute scaling in the cloud, and formal differential privacy throughout the training pipeline.

2. System Architecture

2.1. Overview

The system comprises four architectural layers, each mapped to a distinct academic discipline.

Layer Mapping

CNE Layer	Cloud Network Engineering. Bidirectional gRPC streaming, Kubernetes NetworkPolicy for multi-tenant isolation, ADT debounce rate-limiting.
ECE Layer	Edge Signal Processing. FFT-based spectral seizure detection, importance sampling, bandwidth pruning.
HPC Layer	High-Performance Compute. KEDA event-driven autoscaling, ONNX Runtime inference, multi-threaded gRPC server.
MLOps Layer	Machine Learning Operations. Weighted FedAvg, Opacus differential privacy, MLflow experiment tracking.

2.2. Component Inventory

Table 1: System components and their deployment locations.

Component	Location	Role
Silo clients (×2)	hospital-a, hospital-b ns	FFT edge filter, importance sampler, gRPC stream sender
HPC Aggregator	aggregator ns (1–4 pods)	gRPC server, ONNX inference, Opacus DP training
Global Aggregator	aggregator ns (1 pod)	FedAvg weight averaging, MLflow logging, checkpoint save
MLflow Server	aggregator ns (1 pod)	Experiment tracking, metric storage
Dashboard API	aggregator ns (1 pod)	FastAPI bridge to ADT, K8s, and MLflow REST APIs
Dashboard Frontend	aggregator ns (1 pod)	React SPA served via nginx
Azure Digital Twins	Azure (Southeast Asia)	Real-time patient state mirror
Azure File Storage	Azure (Southeast Asia)	Shared PVC for FL weight exchange between pods
KEDA	keda ns	CPU-triggered autoscaling (1→4 replicas)

2.3. Data Flow

The end-to-end pipeline proceeds as follows.

1. The silo client loads CHB-MIT EEG data and feeds each sample into the FFT edge filter. If the spectral ratio of theta (4–8 Hz) and alpha (8–13 Hz) energy exceeds 0.40, or if a force-burst event fires (8% probability per cycle), the packet is classified as a seizure-risk burst.

2. Non-burst packets are pruned unless the signal amplitude has changed significantly (threshold: 0.05) or a keepalive interval (10 s) has elapsed. This achieves approximately 87% bandwidth reduction.

3. Burst and keepalive packets are streamed to the HPC aggregator over a single persistent bidirectional HTTP/2 gRPC connection.

4. The aggregator runs ONNX Runtime inference on each packet and detects anomalies. On burst packets, it triggers a Opacus-instrumented PyTorch training step, saves privacy-protected weights to the Azure File share, and debounces an Azure Digital Twins update.

5. KEDA monitors CPU utilisation of the aggregator deployment and scales from 1 to 4 replicas when bursts saturate a single pod.

6. Every 30 seconds, the global aggregator reads all weight files from the shared PVC, performs weighted FedAvg across both silos, saves a global checkpoint, evaluates the model, and logs metrics to MLflow.

7. The React dashboard polls the FastAPI bridge every 2 seconds, displaying real-time data from ADT, MLflow, and the Kubernetes API.

3. CNE Upgrade: Bidirectional gRPC Streaming

3.1. Problem with Unary RPC

The V1 prototype used a unary gRPC architecture: each signal packet opened a new TCP connection, sent a single request, received a response, and closed the connection. During a simulated seizure burst, the silo client sent 500 packets in a loop. Each packet incurred a full TCP handshake, TLS negotiation, gRPC framing, and teardown — a total of 500 independent connection cycles per burst event.

Unary gRPC Cost During a 500-Packet Burst

TCP handshakes: 500 × 3-way handshake = 1,500 network round trips
Head-of-line blocking: each request waits for the previous response
Server overhead: 500 separate goroutine contexts spawned and destroyed
Reconnection cost: no stream reuse between normal and burst packets

3.2. Bidirectional Streaming Solution

The upgraded architecture uses a single `StreamSignal` RPC defined in Protocol Buffer 3:

Listing 1: Updated BioNetService proto definition.

```
service BioNetService {
    // One persistent HTTP/2 stream replaces thousands of TCP
       handshakes.
    // Server can push trigger_training signals back to the client.
    rpc StreamSignal (stream SignalRequest) returns (stream
       SignalResponse);
}
```

Each silo client opens one connection per 3,600-second session. The Python generator pattern yields packets into the stream:

Listing 2: Client-side generator feeding the bidi stream.

```
def eeg_signal_generator(eeg_trace, fft_filter):
    while time.time() - stream_start < STREAM_DURATION_SECONDS:
        analysis = fft_filter.push(val)
        is_burst = True if force_burst else analysis["is_burst"]
        if should_send:
            yield biosignal_pb2.SignalRequest(
                hospital_id=HOSPITAL_ID,
                value=val, is_burst=is_burst, timestamp=time.time()
            )
```

The server-side handler iterates lazily over the request iterator and yields one response per packet, enabling real-time back-pressure:

Listing 3: Server-side bidi stream handler.

```
def StreamSignal(self, request_iterator, context):
    for request in request_iterator:
        # ONNX inference runs synchronously
        prob = self.session.run(None, {'input': input_data})[0][0][0]
        # DP training fires asynchronously in a daemon thread
        if request.is_burst:
            self._local_train_step(request.value, label=1.0)
        yield biosignal_pb2.SignalResponse(
            status="PROCESSED", trigger_training=is_anomaly
        )
```

3.3. ADT Debounce

During a seizure burst, 500 packets arrive at the aggregator. Without rate-limiting, each packet would trigger an Azure Digital Twins API call — 500 calls per burst, rapidly exhausting the Azure for Students API quota. A per-silo debounce of 5 seconds limits ADT calls to one per burst regardless of burst duration:

Listing 4: Per-silo debounce protecting ADT API quota.

```
def _debounced_adt_sync(self, twin_id, val, critical):
    now = time.time()
    if now - self._last_adt_sync.get(twin_id, 0) <
       ADT_DEBOUNCE_SECONDS:
        return   # Still within debounce window
    self._last_adt_sync[twin_id] = now
    self.sync_to_azure(twin_id, val, critical)
```

4. ECE Upgrade: FFT-Based Spectral Edge Filter

4.1. Limitation of Amplitude Thresholding

The V1 prototype classified seizure-risk packets using a simple amplitude threshold: `val > 0.8`. This approach is clinically naive. A single muscle artifact or electrode pop produces a transient high-amplitude spike indistinguishable from a seizure onset by amplitude alone. Real seizure detection operates in the frequency domain: seizure activity manifests as sustained rhythmic oscillations in specific EEG frequency bands, not as isolated spikes.

4.2. Frequency Band Physiology

EEG Frequency Bands Relevant to Seizure Detection

Theta (4–8 Hz) Elevated during pre-ictal and ictal states. Sustained theta activity is a marker of focal seizure onset zones.

Alpha (8–13 Hz) Normally suppressed during wakefulness. Paradoxical alpha spikes occur during certain generalised seizure types.

Delta (1–4 Hz) Dominant in slow-wave sleep; also elevated during postictal states. Not used in this implementation but relevant for future extension.

4.3. FFT Filter Implementation

The `FFTEdgeFilter` class maintains a rolling circular buffer of $N = 256$ samples. At 100 Hz this represents 2.56 s of signal, giving a frequency resolution of $100/256 \approx 0.39$ Hz per FFT bin — sufficient to resolve the theta and alpha bands.

Each sample push computes the following pipeline:

$$\text{spectral_ratio} = \frac{\sum_{f\in[4,8]} |\hat{x}(f)| + \sum_{f\in[8,13]} |\hat{x}(f)|}{\sum_{f\in[0,50]} |\hat{x}(f)| + \epsilon} \tag{1}$$

where $\hat{x}(f)$ denotes the magnitude of the real FFT applied to the Hann-windowed buffer. A Hann window is applied before the FFT to reduce spectral leakage at the buffer boundaries:

$$w(n) = 0.5\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right), \quad n = 0, 1, \ldots, N-1 \tag{2}$$

A packet is classified as a seizure-risk burst if spectral_ratio > 0.40, meaning more than 40% of the signal's spectral energy is concentrated in the clinically relevant theta–alpha range.

Table 2: Comparison of amplitude threshold versus FFT spectral filter.

Property	**Amplitude** (> 0.8)	**FFT Spectral Ratio**
Muscle artifact rejection	None	Immune (single-sample events)
Sustained oscillation detection	Partial	Explicit (band energy)
Clinical grounding	None	Theta/alpha physiology
Observable metric	Binary	Continuous (0.0–1.0)
Loggable for analysis	No	Yes (MLflow compatible)
False positive rate	High	Lower

4.4. Bandwidth Reduction

The importance sampling filter (amplitude delta > 0.05 or keepalive interval $> 10\,\mathrm{s}$) achieves approximately **87% bandwidth reduction** in practice, as observed in the silo client logs:

Listing 5: Observed bandwidth efficiency from silo client logs.

```
[PRUNE] Bandwidth Efficiency: 87.6%
[PRUNE] Bandwidth Efficiency: 88.5%
[PRUNE] Bandwidth Efficiency: 80.0%
```

This means only 1 in 8 EEG packets is transmitted over the network. Combined with the bidi-streaming upgrade (one TCP connection vs. 500), the effective network overhead reduction during a burst session is substantial.

5. HPC Upgrade: KEDA Event-Driven Autoscaling

5.1. Scaling Architecture

The HPC aggregator deployment is managed by KEDA (Kubernetes Event-Driven Autoscaler) using a CPU utilisation trigger. The ScaledObject configuration specifies a minimum of 1 replica, a maximum of 4, a 5-second polling interval, and a 15-second cooldown:

Listing 6: KEDA ScaledObject for hpc-aggregator.

```
triggers:
- type: cpu
  metricType: Utilization   # KEDA v2.18 syntax
  metadata:
    value: "1"              # 1% CPU triggers scale-up (demo
       threshold)
```

During a seizure burst, the silo client streams 500 packets in rapid succession. Each packet triggers ONNX inference (compute-bound), saturating a single pod's CPU. KEDA detects the spike within 5 seconds and scales to 4 replicas, distributing the incoming stream across multiple pods via the Kubernetes Service load balancer.

Observed KEDA Scaling in v2-stable

The dashboard confirmed Active pods transitioning from **1 to 4** during burst events, returning to **1** after the 15-second cooldown. KEDA status: `READY=True`, `ACTIVE=True` after fixing the deprecated `type` field to `metricType` in the ScaledObject definition (KEDA v2.18 API change).

5.2. Multi-Tenant Network Isolation

Each hospital silo runs in its own Kubernetes namespace (`hospital-a`, `hospital-b`) with strict NetworkPolicy enforcement. Each silo can only egress to the aggregator namespace — it cannot communicate with the other silo or with external services:

Listing 7: NetworkPolicy restricting hospital-a egress.

```
spec:
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: aggregator
  - ports:
    - protocol: UDP
      port: 53   # DNS only
```

This implements the federated learning principle at the network layer: silos are cryptographically prevented from sharing data with each other, not just assumed to comply.

6. MLOps Upgrade: Real Federated Learning

6.1. From Simulation to Real Backpropagation

The V1 prototype generated random weight vectors to simulate gradient exchange:

Listing 8: V1 simulated weight extraction (replaced).

```
# V1     completely fake
private_weights = (np.random.rand(5) + noise).tolist()
```

The v2 upgrade replaces this with real PyTorch backpropagation. The FFT edge filter's `is_burst` classification serves as a **weak pseudo-label** for supervised training: a packet labelled as a burst receives label $y = 1.0$ (seizure), and the model is trained to distinguish it from normal packets.

Weak Supervision via FFT Labels

This is a form of self-supervised learning: the spectral edge filter on the edge device provides training labels to the central CNN without requiring manual annotation. The edge device's frequency-domain analysis is the oracle; the central model learns to replicate its classification from raw amplitude values alone. This architectural decoupling is a deliberate design choice: the central model can generalise beyond the threshold-based rule once it accumulates enough training examples.

6.2. Weighted Federated Averaging

The global aggregator implements the McMahan et al. (2017) FedAvg algorithm with sample-count weighting:

$$\mathbf{w}_{\text{global}} = \sum_{k=1}^{K} \frac{n_k}{n_{\text{total}}} \mathbf{w}_k \tag{3}$$

where $\mathbf{w}_k$ is the full model `state_dict` from silo k, n_k is the number of training samples from that silo in the current round, and $K = 2$ (Hospital Silo-Alpha and Silo-Beta).

This weight is computed on the actual PyTorch parameter tensors rather than on a simulated 5-element vector, meaning the global model meaningfully aggregates learned representations from both silos.

6.3. Convergence Evidence

The global checkpoint saved to the Azure File PVC after each FedAvg round enables true multi-round FL convergence. The aggregator pod loads this checkpoint on startup, initialising from the last globally aggregated state rather than random weights. Table 3 shows the measured discrimination score across rounds in a representative session.

Table 3: Measured discrimination score across 10 FL rounds (representative session).

Round	Disc. Score	Seizure Conf.	Normal Conf.	Total Samples
1	0.0051	0.526	0.521	66
2	0.0054	0.526	0.521	39
3	0.0054	0.530	0.525	21
4	0.0084	0.543	0.535	69
5	0.0110	0.552	0.541	35
6	0.0135	0.564	0.550	42
7	0.0136	0.566	0.552	10
8	0.0157	0.576	0.560	35
9	0.0186	0.603	0.585	145
10	0.0299	0.624	0.594	–

The monotonically increasing discrimination score confirms that the global model is genuinely learning across rounds, not oscillating around a random initialisation. The low absolute values are expected: the CNN receives a single scalar EEG sample as input rather than a proper time-series window, which fundamentally limits the information available for discrimination. This is an intentional architectural choice at the proof-of-concept stage; extending to windowed input is identified as a future improvement.

7. Security Upgrade: Formal Differential Privacy

7.1. Limitations of Naive Laplacian Noise

The V1 prototype added Laplacian noise to weight vectors post-hoc:

Listing 9: V1 naive DP (replaced in v2).

```
noise = np.random.laplace(0, sensitivity/epsilon, 5)
private_weights = (np.random.rand(5) + noise).tolist()
```

This provides no formal privacy guarantee for two reasons. First, the sensitivity parameter is a guess: without gradient clipping, the true sensitivity of the gradients is unbounded, making the noise scale meaningless. Second, noise is applied after training, not during the backward pass where privacy guarantees must apply.

7.2. Opacus Formal DP Mechanism

The v2 upgrade replaces naive noise with the Opacus library's `PrivacyEngine`, which instruments the PyTorch backward pass directly:

1. **Per-sample gradient computation**: Opacus computes gradients independently for each sample rather than averaging first.
2. **L2 gradient clipping**: each per-sample gradient is clipped to maximum L2 norm $C = 1.0$ before aggregation:

$$\tilde{g}_i = g_i \cdot \min\left(1,\ \frac{C}{\|g_i\|_2}\right) \tag{4}$$

3. **Calibrated Gaussian noise**: noise $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ is added to the clipped gradient sum, where σ (the noise multiplier) is computed by Opacus to achieve the target privacy budget (ε, δ).
4. **Rényi DP accounting**: the exact ε spent per training step is computed using the Rényi Differential Privacy (RDP) accountant and accumulated across steps.

The formal guarantee achieved is $(\varepsilon = 0.5,\ \delta = 10^{-5})$-DP, meaning the probability that a single patient's data is revealed through the model's published weights is bounded above by $e^{0.5} \approx 1.65$ times what it would be without training on that patient.

7.3. Privacy Audit Trail

The per-silo epsilon spent is logged to MLflow as a time-series metric at every aggregation round. Table 4 shows observed values from a representative session, confirming both silos operated within budget.

Table 4: Per-silo differential privacy budget consumption (representative session).

Silo	**ε spent**	**Budget (ε = 0.5)**	**Consumption**
Silo-Alpha	0.2701	0.5	54.0%
Silo-Beta	0.2652	0.5	53.0%

Known Limitation: Opacus-gRPC Threading Conflict

Opacus registers per-sample gradient hooks on the PyTorch model. When `_local_train_step` was called synchronously inside the gRPC generator loop, these hooks fired before gradients were populated due to gRPC's internal threading model, raising a `PerSampleGradient not initialized` exception. The v2-stable fix runs training in a `daemon=True` thread behind a `threading.Lock`, which prevents crashes but adds minor latency. The clean solution (manual gradient clipping replacing Opacus) is documented in `IMPROVEMENTS_GUIDE.md`.

8. Infrastructure Design Decisions

8.1. AKS over Minikube

The V1 prototype ran on local Minikube. An M-series MacBook Air with 8 GB RAM could not sustain KEDA, ONNX Runtime, four aggregator pods, and two silo simulators simultaneously without thermal throttling and memory pressure. Migration to Azure Kubernetes Service (1-node, `standard_b2s_v2`) decoupled compute limits from the development machine.

8.2. Static Azure File PVC

Azure for Students subscriptions enforce HTTPS-only storage policy. Dynamic PVC provisioning via the default `azurefile` storage class failed this policy check. The resolution was to manually create a StorageV2 account with `--https-only true` and bind it using a static PersistentVolume with a pre-created Kubernetes secret containing the storage account key. This gives the PVC `ReadWriteMany` access across all pods in the aggregator namespace, enabling the shared weight buffer between the HPC aggregator and global aggregator deployments.

8.3. MLflow Version Pinning

MLflow was pinned to v2.8.0 rather than `:latest` because newer versions introduced strict Host Header validation that rejected Kubernetes internal DNS names (`*.svc.cluster.local`) with a 403 DNS rebinding error. The pinned version accepts these names, enabling the aggregator to log metrics without port-forwarding.

8.4. JSON Patch Operation

Azure Digital Twins property updates use the `"op": "add"` operation rather than `"op": "replace"` to ensure idempotency. A fresh twin instance has no existing property values; `replace` on a non-existent path throws a 404. `add` creates the property if absent and updates it if present, making the sync operation safe regardless of twin initialisation state.

9. Measured Results

Table 5: Verified performance metrics from v2-stable deployment on AKS.

Metric	**Observed Value**	**How Measured**
Bandwidth reduction	∼87%	Packets saved / total attempts, logged by silo client
TCP connections per 500-packet burst	1 (was 500)	Bidirectional stream vs. unary per-packet
Pod scaling on burst	1 → 4 replicas	Dashboard KEDA panel, confirmed via `kubectl get pods -w`
FL rounds (10-min session)	20+	MLflow experiment run count
Discrimination score trend	Monotonically increasing	MLflow metrics across 20 consecutive rounds
Privacy budget (Silo-Alpha)	$\varepsilon = 0.2701$ of 0.5	Opacus Rényi DP accountant, logged to MLflow
Privacy budget (Silo-Beta)	$\varepsilon = 0.2652$ of 0.5	Opacus Rényi DP accountant, logged to MLflow
ADT update latency	$\leq$ 5 s (debounced)	Azure Digital Twins Explorer live observation
Pod restart count (v2-stable)	0	`kubectl get pods -n aggregator`
Cluster cost (active)	∼$0.05/hr	Azure for Students billing dashboard

10. Dashboard and Observability

10.1. Architecture

The observability layer comprises two containers in the aggregator namespace: a React single-page application (SPA) served by nginx, and a FastAPI bridge that polls the Azure and Kubernetes APIs.

Dashboard Data Sources

/api/adt	Azure Digital Twins SDK: HeartRate and IsCritical properties for Silo-Alpha and Silo-Beta.
/api/pods	Kubernetes `AppsV1Api`: ready replica count for the hpc-aggregator deployment.
/api/mlflow	MLflow REST API: discrimination score, seizure and normal confidence, training samples per round.
/api/dp	MLflow REST API: per-silo epsilon spent from the most recent completed run.
EEG waveform	Simulated in the frontend at 2 Hz; WebSocket real-time streaming is identified as a future improvement.

10.2. Five Live Panels

1. **EEG signal stream**: Two charts — raw amplitude (0–1 normalised) and FFT spectral ratio with threshold line at 0.40. Both charts update every 500 ms. During a seizure event, the amplitude chart turns red and a "Theta burst detected" badge appears.
2. **FL convergence**: Line chart of discrimination score, seizure confidence, and normal confidence across FL rounds, sourced from MLflow runs. The current discrimination score is shown as a large metric above the chart.
3. **Privacy budget**: Two gauge bars, one per silo, showing epsilon consumed as a percentage of the budget. Bars turn amber at 60% and red at 80%, with a warning badge recommending aggregation.
4. **Azure Digital Twins**: Two cards showing live HeartRate and IsCritical values. Card borders turn red during a seizure event.
5. **KEDA autoscaling**: Pod count with four visual slot indicators. Slots illuminate blue as pods scale up during a burst.

11. Known Limitations and Honest Assessment

11.1. What This Project Is

Bio-Sync HPC is a **distributed systems and MLOps engineering project** that uses EEG seizure detection as its domain problem. The primary academic and professional contribution is the infrastructure architecture, not the clinical model performance. This framing is important for honest evaluation.

11.2. Model Performance Limitation

The 1D-CNN receives a single scalar EEG amplitude value as input. A single amplitude sample contains essentially no clinically useful information for seizure classification — this

requires a multi-sample time window to resolve frequency content. The discrimination score of ∼0.03 is therefore expected and does not indicate a flaw in the FL pipeline itself; it reflects an input design that is appropriate for a proof-of-concept infrastructure demonstration but insufficient for clinical use.

The correct next step (documented in `IMPROVEMENTS_GUIDE.md`) is to transmit the full 256-sample FFT buffer per packet, change the model input shape from $[1, 1, 1]$ to $[1, 1, 256]$, and measure accuracy against CHB-MIT labelled seizure segments.

11.3. What Can Be Honestly Claimed

Defensible Claims for Academic and Professional Contexts

1. **CNE**: Bidirectional gRPC reduces TCP connection overhead from $O(n)$ to $O(1)$ during seizure burst events; demonstrated on AKS.
2. **ECE**: FFT-based importance sampling achieves ∼87% bandwidth reduction compared to sending all samples, measured in live logs.
3. **HPC**: KEDA autoscaling from 1 to 4 pods observed on CPU burst trigger; confirmed READY=True in production.
4. **Security**: Formal $(\varepsilon = 0.5, \delta = 10^{-5})$-DP guarantee via Opacus; per-silo epsilon tracked and auditable in MLflow.
5. **MLOps**: Real weighted FedAvg on actual PyTorch gradients; monotonically increasing discrimination score over 20+ rounds.

12. Future Work

1. **Manual DP replacing Opacus**: Implement L2 gradient clipping and Gaussian noise injection directly in PyTorch without Opacus, eliminating the threading conflict entirely.
2. **Windowed CNN input**: Transmit the 256-sample FFT buffer per packet over gRPC using a `repeated double` proto field. Regenerate the ONNX model with input shape $[1, 1, 256]$. Measure true positive and false positive rates against labelled CHB-MIT seizure segments to produce clinical accuracy metrics.
3. **Flower FL framework**: Replace the JSON file-based weight exchange and polling loop with the Flower federated learning protocol, enabling proper client selection, round management, and strategy selection (FedProx, FedAdam).
4. **mTLS for gRPC**: Add mutual TLS to the gRPC channel, using Kubernetes Secrets to distribute certificates. This makes the network layer production-grade and directly demonstrates CNE security engineering.
5. **Real-time EEG WebSocket**: Replace the simulated EEG waveform in the dashboard with a WebSocket endpoint in the FastAPI bridge, fed by the HPC aggregator forwarding real packet values.

6. **SpectralRatio in ADT**: Add `SpectralRatio` as a property to the DTDL patient model and sync the FFT output to ADT on each burst, making it visible in the dashboard and Digital Twins Explorer.

13. Conclusions

Bio-Sync HPC demonstrates that a cross-layer approach to federated learning for biomedical IoT is both architecturally achievable and practically deployable at student-subscription scale. The four upgrades from V1 to v2-stable each address a concrete engineering problem with a measurable outcome: bidirectional streaming eliminates per-packet connection overhead; FFT-based filtering reduces bandwidth consumption by 87%; KEDA autoscaling responds to seizure bursts within 5 seconds; and formal differential privacy provides an auditable privacy guarantee compliant with clinical data regulations.

The infrastructure is fully deployed, observable, and stable. The primary remaining gap is clinical model accuracy, which requires a windowed input representation and is well-understood with a clear implementation path.

Repository and Deployment

The complete source code, including all Kubernetes manifests, Docker configurations, and the React dashboard, is version-controlled at git tag `v2-stable`. The live dashboard is accessible at `http://20.198.144.247` while the AKS cluster is running. Azure resource group: `BioNet-HPC-RG`, region: Southeast Asia.